Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-208БВ-25

Студент: Игнатович П.В.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 17.10.26

Москва, 2026

Постановка задачи

Вариант 9.

Цель работы: приобретение практических навыков управления процессами в ОС и обеспечения обмена данными между процессами посредством каналов.

Родительский процесс создаёт дочерний процесс. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое используется для открытия файла на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Стандартный поток вывода дочернего процесса перенаправляется в pipe1. Родительский процесс читает из pipe1 и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процессы должны быть представлены разными программами.

В файле записаны команды вида: «число число число<endline>». Дочерний процесс производит деление первого числа команды на последующие числа в команде, а результат выводит в стандартный поток вывода. Если происходит деление на 0, то дочерний и родительский процессы завершают свою работу. Проверка деления на 0 должна осуществляться на стороне дочернего процесса. Числа имеют тип float. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- pid_t fork(void); – создаёт дочерний процесс (копию родителя).
- int execv(const char *path, char *const argv[]); – заменяет образ памяти дочернего процесса программой child.
- int pipe(int pipefd[2]); – создаёт неименованный канал pipe1 (pipefd[0] – чтение, pipefd[1] – запись).
- int dup2(int oldfd, int newfd); – переназначает файловый дескриптор: stdin ← открытый файл, stdout ← pipe1.
- int open(const char *pathname, int flags, mode_t mode); – открывает файл с командами на чтение (O_RDONLY).
- int close(int fd); – закрывает ненужные дескрипторы (в том числе концы канала).
- ssize_t read(int fd, void *buf, size_t count); / ssize_t write(...); – ввод-вывод через дескрипторы.
- pid_t waitpid(pid_t pid, int *status, int options); – ожидание завершения дочернего процесса и получение кода возврата.
- void exit(int status); – завершение процесса с кодом возврата.
- ssize_t readlink(const char *path, char *buf, size_t size); – получение пути к исполняемому файлу через /proc/self/exe, чтобы найти программу child рядом с родителем.

Работа состоит из двух программ: parent.c (родительский процесс) и child.c (дочерний процесс).

Родительский процесс выводит приглашение, считывает с клавиатуры первую строку (имя файла) и открывает файл вызовом open(O_RDONLY). Затем создаётся канал pipe1 и

вызовом fork() порождается дочерний процесс. В дочернем процессе закрывается неиспользуемый конец канала на чтение, после чего с помощью dup2 стандартный ввод заменяется открытым файлом, а стандартный вывод – концом канала pipe1 на запись. Исходные дескрипторы закрываются, и вызовом execv запускается программа child.

Дочерняя программа не знает ни о файле, ни о канале: она читает данные из stdin блоками через read() и выделяет из них строки по символу перевода строки (строка может приходить частями, поэтому используется собственный буфер). Каждая строка разбирается функцией strtof() на числа типа float. Результат – первое число, последовательно делённое на все остальные. Перед каждым делением выполняется проверка делителя на 0: при нулевом делителе дочерний процесс пишет сообщение в stderr и завершается с кодом EXIT_FAILURE. Если деления не было (в строке одно число) – выводится само число. Результат каждой команды записывается в stdout (то есть в pipe1) отдельной строкой.

Родительский процесс закрывает конец канала на запись и в цикле читает pipe1, выводя полученное в свой стандартный вывод, пока не наступит конец файла (все копии записывающего конца закрыты – то есть дочерний процесс завершился). После этого вызывается waitpid(); если дочерний процесс завершился с ненулевым кодом (деление на 0, ошибка разбора), родительский процесс сообщает об этом и тоже завершается с ошибкой. Системные ошибки (open, pipe, fork, dup2, execv, read, write, waitpid) проверяются и обрабатываются выводом сообщения в stderr и завершением работы.

Код программы

parent.c

```
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define CHILD_PROGRAM_NAME "child"

static void print_err(const char *msg) {
    write(STDERR_FILENO, msg, strlen(msg));
}

int main(void) {
    // Определяем каталог, в котором лежит исполняемый файл родителя
    char progpath[1024];
    ssize_t plen = readlink("/proc/self/exe", progpath, sizeof(progpath) - 1);
    if (plen == -1) {
        print_err("error: failed to read full program path\n");
        exit(EXIT_FAILURE);
    }
    progpath[plen] = '\0';
    while (plen > 0 && progpath[plen] != '/')
        --plen;
    progpath[plen] = '\0';

    // Первая строка: имя файла
    const char prompt[] = "Enter file name: ";
    write(STDOUT_FILENO, prompt, sizeof(prompt) - 1);
```

```
char fname[1024];
size_t flen = 0;
for (;;) {
    char c;
    ssize_t n = read(STDIN_FILENO, &c, 1);
    if (n < 0) {
        print_err("error: failed to read file name from stdin\n");
        exit(EXIT_FAILURE);
    }
    if (n == 0 || c == '\n')
        break;
    if (flen < sizeof(fname) - 1)
        fname[flen++] = c;
}
fname[flen] = '\0';
if (flen == 0) {
    print_err("error: empty file name\n");
    exit(EXIT_FAILURE);
}

// Родитель открывает файл на чтение
int file = open(fname, O_RDONLY);
if (file == -1) {
    print_err("error: failed to open requested file\n");
    exit(EXIT_FAILURE);
}

// pipe1: child (stdout) -> parent
int pipe1[2];
if (pipe(pipe1) == -1) {
    print_err("error: failed to create pipe\n");
    exit(EXIT_FAILURE);
}

const pid_t child = fork();
if (child == -1) {
    print_err("error: failed to spawn new process\n");
    exit(EXIT_FAILURE);
}

if (child == 0) {
    // Дочерний процесс: stdin <- файл, stdout -> pipe1
    close(pipe1[0]);

    if (dup2(file, STDIN_FILENO) == -1 ||
        dup2(pipe1[1], STDOUT_FILENO) == -1) {
        print_err("error: failed to redirect standard streams\n");
        exit(EXIT_FAILURE);
    }
    close(file);
    close(pipe1[1]);

    char path[2048];
    snprintf(path, sizeof(path), "%s/%s", progpath, CHILD_PROGRAM_NAME);
    char *const args[] = {CHILD_PROGRAM_NAME, NULL};
    execv(path, args);

    print_err("error: failed to exec into new executable image\n");
    exit(EXIT_FAILURE);
}

// Родитель
close(file);
close(pipe1[1]);

char buf[4096];
ssize_t bytes;
while ((bytes = read(pipe1[0], buf, sizeof(buf))) != 0) {
    if (bytes < 0) {
```

```
            if (errno == EINTR)
                continue;
            print_err("error: failed to read from pipe\n");
            exit(EXIT_FAILURE);
        }
        if (write(STDOUT_FILENO, buf, bytes) != bytes) {
            print_err("error: failed to write to stdout\n");
            exit(EXIT_FAILURE);
        }
    }
    close(pipe1[0]);

    int status = 0;
    if (waitpid(child, &status, 0) == -1) {
        print_err("error: failed to wait for child\n");
        exit(EXIT_FAILURE);
    }
    if (WIFEXITED(status) && WEXITSTATUS(status) != EXIT_SUCCESS) {
        print_err("parent: child terminated with error, exiting\n");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

child.c

```
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>

static void print_err(const char *msg) {
    write(STDERR_FILENO, msg, strlen(msg));
}

// Обрабатывает одну команду «число число число». Возвращает 0 при успехе,
// -1 при делении на 0, -2 при ошибке разбора.
static int process_line(char *line) {
    float result = 0.0f;
    int count = 0;

    char *p = line;
    while (*p) {
        char *end;
        errno = 0;
        float v = strtof(p, &end);
        if (end == p) {
            if (*p == ' ' || *p == '\t' || *p == '\r') {
                ++p;
                continue;
            }
            return -2;
        }
        if (count == 0) {
            result = v;
        } else {
            // Проверка деления на 0 выполняется в дочернем процессе
            if (v == 0.0f)
                return -1;
            result /= v;
        }
        ++count;
        p = end;
    }
    if (count == 0)
        return 0; // пустая строка — пропускаем
```

```
    char out[128];
    int len = snprintf(out, sizeof(out), "%g\n", result);
    if (write(STDOUT_FILENO, out, len) != len) {
        print_err("error: failed to write to stdout\n");
        exit(EXIT_FAILURE);
    }
    return 0;
}

int main(void) {
    char buf[4096];
    char line[8192];
    size_t llen = 0;
    ssize_t bytes;

    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) != 0) {
        if (bytes < 0) {
            if (errno == EINTR)
                continue;
            print_err("error: failed to read from stdin\n");
            exit(EXIT_FAILURE);
        }
        for (ssize_t i = 0; i < bytes; ++i) {
            if (buf[i] != '\n') {
                if (llen < sizeof(line) - 1)
                    line[llen++] = buf[i];
                continue;
            }
            line[llen] = '\0';
            llen = 0;
            int r = process_line(line);
            if (r == -1) {
                print_err("child: error: division by zero\n");
                exit(EXIT_FAILURE);
            } else if (r == -2) {
                print_err("child: error: invalid number in command\n");
                exit(EXIT_FAILURE);
            }
        }
    }
    if (llen > 0) { // последняя строка без \n
        line[llen] = '\0';
        int r = process_line(line);
        if (r != 0) {
            print_err(r == -1 ? "child: error: division by zero\n"
                              : "child: error: invalid number in command\n");
            exit(EXIT_FAILURE);
        }
    }
    return 0;
}
```

Сборка

```
gcc -Wall -Wextra -o parent parent.c
gcc -Wall -Wextra -o child child.c
```

Протокол работы программы

Тестирование:

Тест 1. Корректные команды (результат: 10/2/5=1; 7.5/2.5=3; 100/4/5/2=2.5; одно число 3.5 выводится как есть).

```
$ cat test1.txt
10 2 5
7.5 2.5
100 4 5 2
```

```
3.5
$ ./parent
Enter file name: test1.txt
1
3
2.5
3.5
$ echo $?
0
```

Тест 2. Деление на 0 в первой же команде: дочерний и родительский процессы завершаются.

```
$ cat test2.txt
10 2 0
5 5
$ ./parent
Enter file name: test2.txt
child: error: division by zero
parent: child terminated with error, exiting
$ echo $?
1
```

Тест 3. Деление на 0 во второй команде: первая команда успевает обработаться, затем оба процесса завершаются.

```
$ cat test3.txt
8 2
9 3 0 1
1 1
$ ./parent
Enter file name: test3.txt
4
child: error: division by zero
parent: child terminated with error, exiting
$ echo $?
1
```

Тест 4. Несуществующий файл (обработка системной ошибки open).

```
$ ./parent
Enter file name: nofile.txt
error: failed to open requested file
$ echo $?
1
```

Тест 5. Дробные числа и отрицательные значения.

```
$ cat test5.txt
1 0.5 0.25
-9 3
2.5 -0.5
$ ./parent
Enter file name: test5.txt
8
-3
-5
$ echo $?
0
```

Вывод

В ходе лабораторной работы были реализованы две программы на языке Си – родительская и дочерняя, взаимодействующие через неименованный канал, с переопределением стандартных потоков ввода и вывода дочернего процесса с помощью dup2. Были освоены системные вызовы fork, execv, pipe, dup2, open, close, waitpid, а также правила работы с каналами: необходимость закрывать неиспользуемые концы канала, чтобы

читающая сторона получила признак конца данных. Основная сложность – корректная буферизация при чтении строк блоками через read() и передача информации об ошибке (делении на 0) от дочернего процесса родительскому через код возврата, получаемый в waitpid. Все системные ошибки обрабатываются, поставленная задача варианта 9 выполнена.